Classification and its metrics

MNIST · detecting one digit

LECTURE 3 | GÉRON CH. 3

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Two lectures on a regression problem, ending with a number and an interval around it:

California housing	RMSE
Predict a constant — the baseline	$115,311
Best honest cross-validated estimate	$48,180
Final, on the test set, once	$49,037

> **THE STANDING CONSTRAINT, UNCHANGED**
>
> Split before anything is fitted. All preprocessing inside a `Pipeline` passed to cross-validation. Nothing derived from the test set in the training path. Fixed seeds. Report the fold scores, not only the mean.

Today the problem is classification, the metric is not RMSE, and the baseline turns out to matter more than it did in either of the last two lectures.

Today

1. The task, the data, and a first detector (15 min)
2. **The mathematics** — why accuracy fails under imbalance, and why precision is not monotone in the threshold (20 min)
3. **The method** — the confusion matrix, precision, recall, F1, the trade-off, PR and ROC (40 min)
4. Choosing an operating point, and beyond binary (15 min)

One number runs through the whole lecture. It is correctly measured, and it is worth almost nothing — and the second block is what lets you say why.

FIRST FIFTEEN MINUTES

The task, and a first detector

15 minutes

You have been hired again

A national postal operator sorts handwritten postcodes automatically. A camera photographs each digit; a classifier reads it; the envelope goes down a chute.

> **THE TASK**
>
> Their audit team has found that one glyph is misread far more often than the others. They want every scanned digit that *is* a **5** pulled off the line and sent to a human verification desk.

Not “read the digit”. **Detect one digit.** Yes or no, one class against the other nine.

Framing, answered

Question	Answer
Supervision	Supervised — every image carries a label
Task	**Binary classification**: 5 or not-5
Learning	Batch. The corpus is fixed and fits in memory
Assumption	Next year's handwriting looks like this corpus

The fourth is the one that will age. Handwriting drifts, cameras are replaced, and a model trained on one decade's post is not automatically valid on the next.

✗ Classification, not regression. So RMSE is gone, and we need a new metric. That is the whole reason this application exists.

What makes this a *rare-event* problem

The ten digits appear in roughly equal numbers. The task does not.

$$\text{one class against nine} \implies \text{about one positive in ten}$$

Every binary detector carved out of a multiclass problem inherits this. Detect one product fault out of forty; one disease out of a hundred; one fraudulent transaction in ten thousand. The arithmetic is the same and only the ratio changes.

The data: MNIST

- 70,000 small images of handwritten digits, written by US high-school students and Census Bureau employees
- Each image is 28 × 28 pixels, flattened to **784** features
- Each feature is one pixel's intensity, an integer from 0 (white) to 255 (black)
- Every image is labelled with the digit it shows

> *The textbook calls MNIST "the hello world of machine learning". It is studied so often that it is effectively the field's control experiment: if a method cannot do something here, the something is probably not the method's fault.*

The first default nobody asked for

`y` arrives as an array of **one-character strings**: `'5'`, not `5`.

```
y = y.astype(np.uint8)                # do this before anything else

assert y.dtype == np.uint8
assert set(np.unique(y)) == set(range(10))
```

WHAT IT COSTS IF YOU FORGET

`y == 5` is then `False` for every single image, with no error and no warning — a string is never equal to an integer. You would build a detector for a class that has no members and score **100%** accuracy on it.

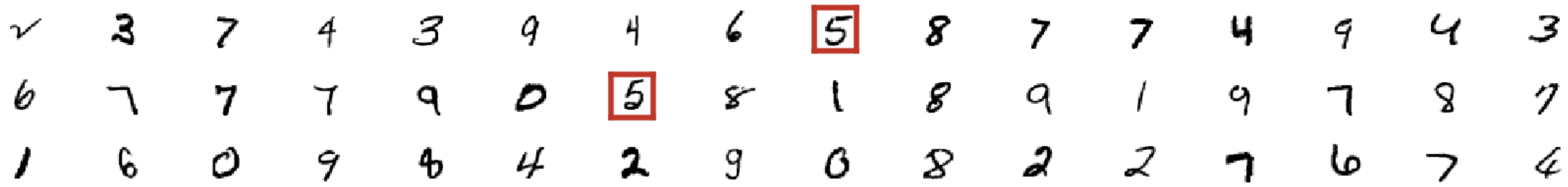
A label compared against the wrong type

FAILURE CONDITION

`y == 5` is `False` for every image when the labels are strings: a string is never equal to an int, and neither NumPy nor Scikit-Learn objects. The model then trains on a target that is false everywhere and reports the accuracy of predicting one class.

Forty-eight of the seventy thousand

48 digits drawn at random — 2 of them are 5s, ringed in red



Drawn at random, and exactly **2** of the forty-eight are 5s. That ratio is the whole problem.

The split has already been made for us

```
X_train, X_test = X[:60000], X[60000:]
y_train, y_test = y[:60000], y[60000:]

assert len(X_train) == 60000 and len(X_test) == 10000
assert X_train.shape[1] == X_test.shape[1] == 784
```

- The first 60,000 rows are the standard training set; the last 10,000 are the standard test set
- The corpus is **already shuffled**, so the first 60,000 are not sorted by digit or by writer
- Using the standard split is what makes any published number on MNIST comparable to ours

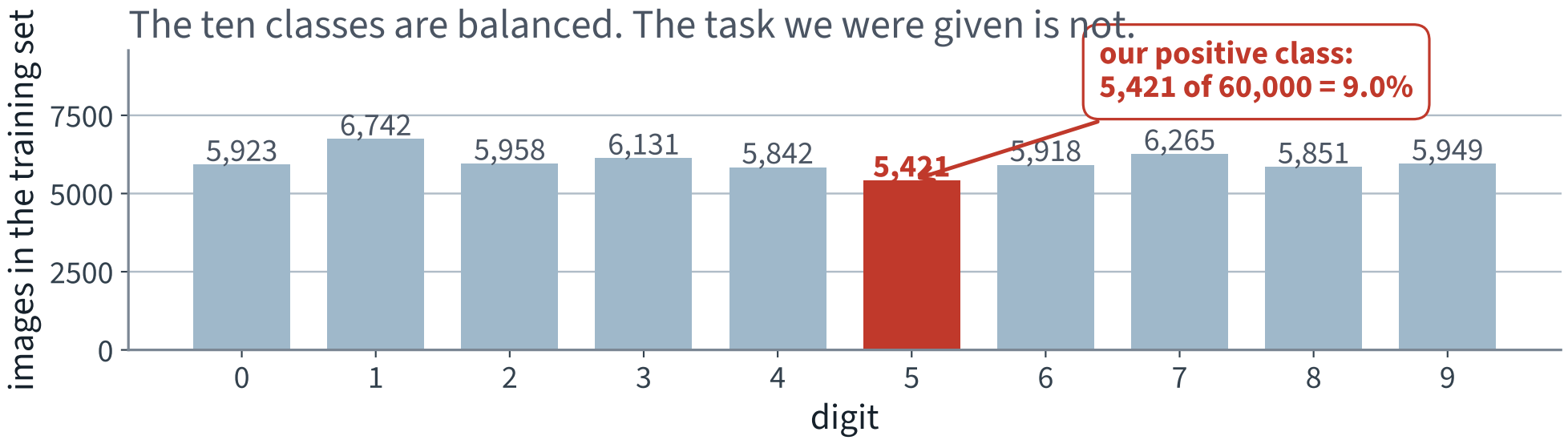
From ten classes to two

```
y_train_5 = (y_train == 5)          # True for every 5, False otherwise
y_test_5  = (y_test  == 5)

assert y_train_5.dtype == bool
assert y_train_5.sum() == 5421
print(f"{y_train_5.sum():,} positives out of {len(y_train_5):,}")
```

Two lines, and the imbalance is created. Nothing about MNIST is imbalanced; **our question is**.

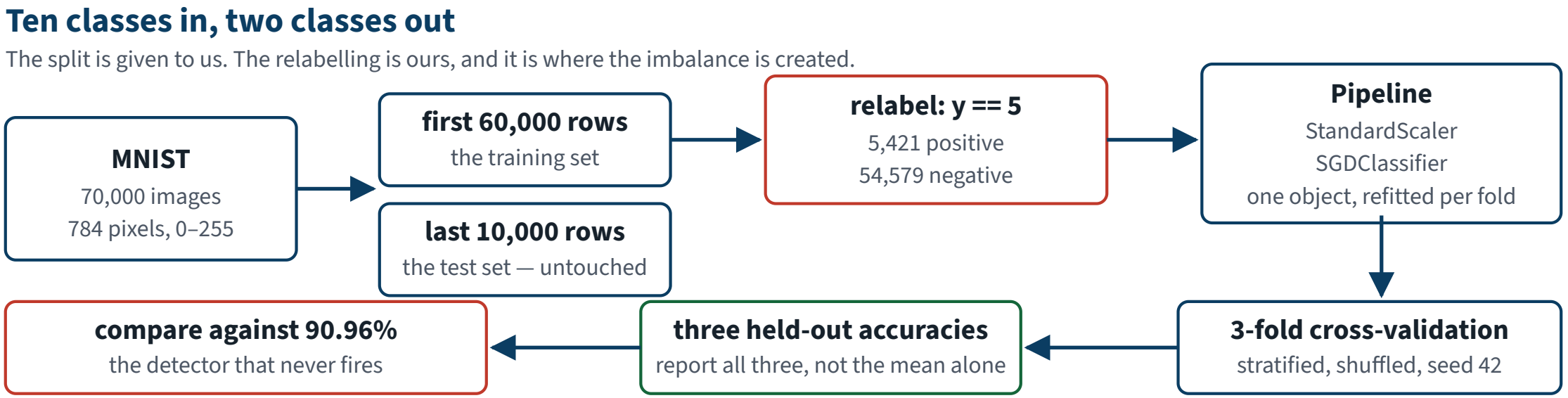
✗ Assert the count. If the cast to `uint8` was skipped, this assertion is what tells you — it fails at `0 == 5421` instead of silently training on nothing.

Balanced data, imbalanced question



Every bar is roughly the same height. The moment we ask “is it a 5?”, one bar becomes the positive class and the other nine become the negative class.

The whole build, before we write it



Six boxes. The red one is where the imbalance is manufactured, and the red one on the left is what every result has to be compared against.

Metric before model

A habit this course returns to, and we are about to apply it for the second time.

- Choose what you are optimising **before** you have anything to optimise
- Otherwise the metric gets chosen after the result, which is how you end up reporting whichever number happens to look best

✗ And name who chose it. If the metric arrived by default — from a library, from a habit, from an assistant — then nobody chose it.

The obvious metric for a classifier

Accuracy: the fraction of instances the classifier gets right.

$$\text{accuracy} = \frac{\#\{\, i : \hat{y}^{(i)} = y^{(i)} \,\}}{m}$$

THE PROPORTION OF CORRECT PREDICTIONS

- It is between 0 and 1, and bigger is better
- It needs no threshold, no scaling and no interpretation
- Every stakeholder in the world understands it without being taught

Build it

```
from sklearn.linear_model import SGDClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

clf = make_pipeline(StandardScaler(),
                    SGDClassifier(random_state=42))

clf.fit(X_train, y_train_5)
```

Two components, one object. The standing constraint is satisfied by construction: when cross-validation holds out a fold, the scaler is refitted on the other folds only.

`random_state=42` because SGD shuffles the training instances. Without it, two runs of the same code give different models — and you will spend an afternoon on the difference.

The headline number

96.90%

Three-fold cross-validated accuracy, on data the model never saw during its own fitting.

Folds: 96.83%, 96.84%, 97.04%. A spread of about two tenths of a point — the three folds agree, which is more than could be said for anything in the previous application.

Before you commit: what does *nothing* score?

The cheapest possible detector. It looks at nothing and answers “not a 5” every time.

```
from sklearn.base import BaseEstimator

class NeverFires(BaseEstimator):
    def fit(self, X, y=None):
        return self
    def predict(self, X):
        return np.zeros(len(X), dtype=bool)

cross_val_score(NeverFires(), X_train, y_train_5, cv=cv,
                scoring="accuracy")     # array([0.90965, 0.90965, 0.90965])
```

✓ It is right 54,579 times out of 60,000 — 90.96% accuracy, with no model, no fit and no features.

How to read that

- First, what it *is*: with no positive prediction, accuracy is **exactly the negative class's base rate** — 90.96% is the share of the training set that is not a 5. An identity about the data, with no model in it
- Only then, what it means: that accuracy is available for free, so 90.96% is **zero** in the units that matter
- The interesting quantity is the distance above it, not the number itself
- The anchor is different for every task — 88.76% for digit 1, because there are more 1s in the corpus than any other digit

> **RULE 2, APPLIED HERE**
>
> A metric with nothing to compare it to is decoration. You measured a constant predictor before committing an RMSE; measure a constant classifier before committing an accuracy.

Against the anchor

Detector	Cross-validated accuracy
Never fires — the anchor	90.96%
SGD on raw pixels	95.03%
SGD in a pipeline with scaling	**96.90% ✓**
Perfect	100%

Our detector is **5.94 percentage points** above doing nothing, and 3.10 points below perfect. Of the mistakes the anchor makes, we have removed **65.7%**.

Both of those are the same measurement stated two ways. Say which one you mean when you report it.

THE MATHEMATICS

Imbalance, and the non-monotonicity of precision

20 minutes · examinable

One question, for the whole lecture

Doing nothing scores **90.96%**.

We score **96.90%**.

✗ What, exactly, did those 5.94 points buy?

The number is correctly cross-validated, there is no leakage in it, and the three folds agree to two tenths of a point. Everything about how it was measured is right. That is what makes the question worth twenty minutes.

Two claims, both provable

1. A classifier that **never fires** attains an accuracy of exactly one minus the base rate — and it is not an approximation, it is an identity
2. As the decision threshold rises, **recall is monotone and precision is not** — because recall's denominator is fixed and precision's is not

> **WHY YOU NEED THEM**
>
> The first tells you what your accuracy is worth. The second tells you why you cannot simply turn a dial until both numbers are good.

Notation, fixed for the whole derivation

- m instances, each with a true label $y^{(i)} \in \{0, 1\}$
- P — the number of positives, N — the number of negatives, $P + N = m$
- $p = P/m$ — the **base rate**, or prevalence
- A scoring function s and a threshold t; predict positive when $s(x) \geq t$
- $\mathrm{TP}(t), \mathrm{FP}(t), \mathrm{FN}(t), \mathrm{TN}(t)$ — the four counts at threshold t

Everything below is arithmetic on those four counts. There is no probability theory in this derivation and none is needed.

Accuracy, in the four counts

$$\text{accuracy}(t) = \frac{\text{TP}(t) + \text{TN}(t)}{m}$$

Two of the four counts in the numerator, all four in the denominator. Note that the two mistakes — FP and FN — enter with **equal weight**.

✗ A missed 5 and a wrongly flagged 3 cost the same, as far as this formula is concerned. Nobody in the postal operator believes that.

Claim 1 · the classifier that never fires

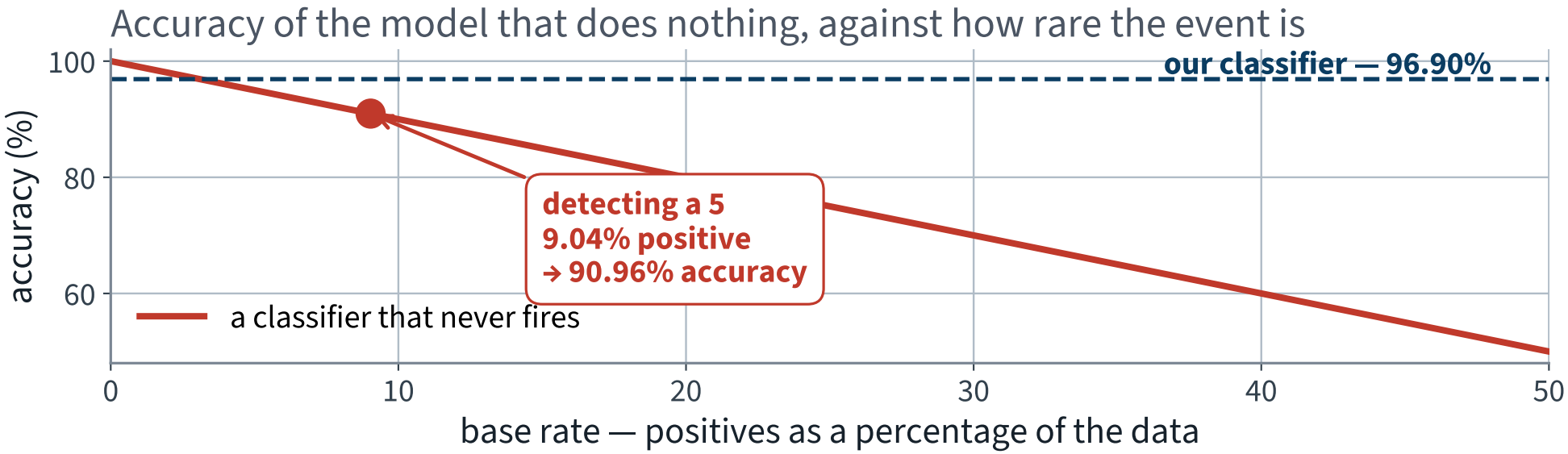
Let $h_0(x) = 0$ for every x. Then $\mathrm{TP} = 0$ and $\mathrm{FP} = 0$, so every negative is a true negative:

$$\mathrm{accuracy}(h_0) \;=\; \frac{0 + N}{m} \;=\; \frac{N}{m} \;=\; 1 - p$$

AN IDENTITY, NOT AN ESTIMATE — NO MODEL, NO DATA, NO VARIANCE

On our task, $p = 0.09035$, so $\mathrm{accuracy}(h_0) = 0.90965$. That is the 90.96% you measured, and you could have written it down without running anything.

The accuracy of doing nothing



✗ The rarer the event, the better the model that does nothing looks. At a base rate of 1%, doing nothing scores 99%.

A metric with nothing to compare it to

FAILURE CONDITION

The rarer the event, the better the model that does nothing looks. Without the base rate beside it an accuracy cannot be located against $1 - p$, so a result that beats doing nothing by a tenth of a point reads exactly like one that beats it by thirty.

Corollary · accuracy is unbounded above by uselessness

$$\lim_{p \to 0^+} \text{accuracy}(h_0) \;=\; 1$$

For any target accuracy $a < 1$ you care to name, there is a detection problem rare enough that the empty model beats it.

> So “99.2% accurate” is not a statement about a classifier. It is a statement about a classifier *and* a base rate, and quoting it without the second is quoting half a measurement.

That is this lecture’s specimen question, answered — and the shape of a Part C question on the paper.

Where the rest of accuracy goes

Split the four counts by their true class. Write $\text{recall} = \text{TP}/P$ and $\text{specificity} = \text{TN}/N$. Then

$$\begin{aligned}\text{accuracy} &= \frac{\text{TP} + \text{TN}}{m} = \frac{P}{m} \cdot \frac{\text{TP}}{P} + \frac{N}{m} \cdot \frac{\text{TN}}{N} \\ &= p\,\text{recall} + (1 - p)\,\text{specificity}\end{aligned}$$

ACCURACY IS A WEIGHTED AVERAGE OF TWO RATES, WEIGHTED BY THE CLASS SIZES

✓ This is the whole of claim 1, and claim 1 is the special case $\text{recall} = 0$**,** $\text{specificity} = 1$**.**

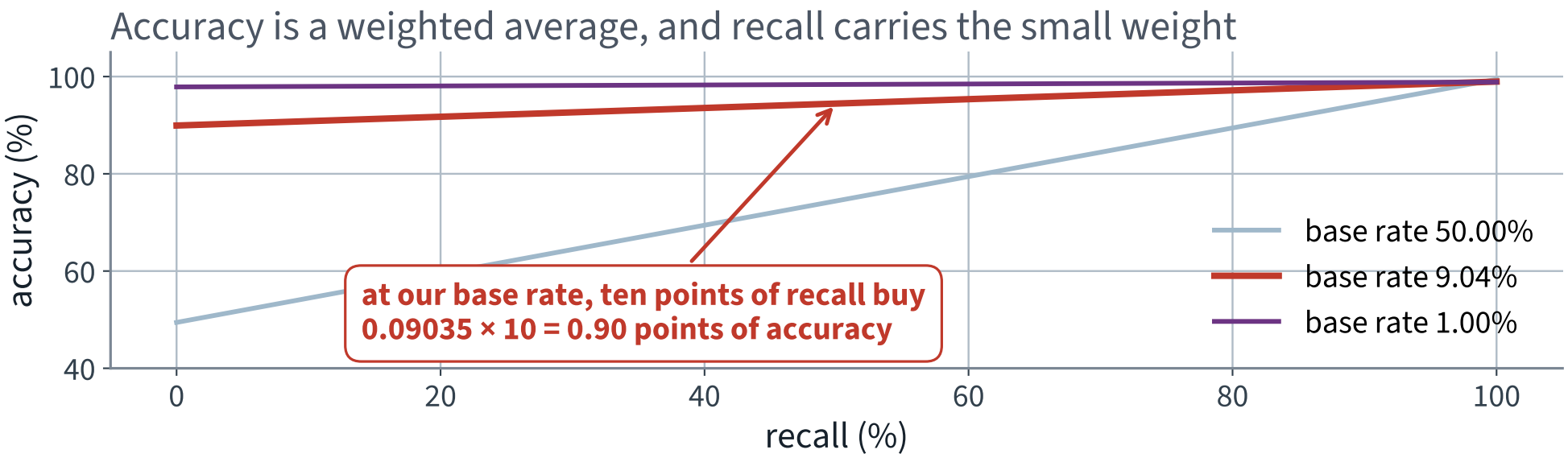
The identity, on our own detector

Term	Value	Contribution to accuracy
$(1-p)\cdot\text{specificity}$	0.90965 × 0.98860	0.89928
$p\cdot\text{recall}$	0.09035 × 0.77181	**0.06973 ✓**
accuracy	—	0.96902

✗ 92.8% of our headline number is the first row — being right about the digits that are *not* 5s.

Read that carefully. Everything the detector does about the class we were hired to find is the 0.0697 on the second row.

What a point of recall is worth



The slope of each line *is* the base rate. At ours, ten points of recall move accuracy by 0.90 points.

Stated as a derivative

$$\frac{\partial\,\text{accuracy}}{\partial\,\text{recall}} = p \qquad \frac{\partial\,\text{accuracy}}{\partial\,\text{specificity}} = 1 - p$$

Accuracy responds to the class you care about in proportion to how little of it there is.

THE CONSEQUENCE, STATED PLAINLY

You can lose **a third of the 5s** and pay under three points of accuracy for it. If accuracy is what you optimise, that trade looks attractive.

Part two · what happens when you turn the dial

Every scoring classifier is really a family of classifiers, one for each threshold t:

$$\hat{y}_t(x) \;=\; \mathbf{1}\big[\, s(x) \geq t \,\big]$$

Raising t makes the classifier more reluctant to fire. Lowering it makes the classifier more eager. Nothing about the model changes — only where we cut.

✓ So there is a dial. The question is what the two metrics do as we turn it.

What is monotone, immediately

The flagged set $\{x : s(x) \geq t\}$ shrinks as t rises, and it shrinks by *losing* instances, never by gaining any. So for $t' \geq t$:

$$\mathrm{TP}(t') \leq \mathrm{TP}(t), \qquad \mathrm{FP}(t') \leq \mathrm{FP}(t)$$

Both are non-increasing. Every quantity here is built from these two.

Recall is monotone, and here is why

$$\text{recall}(t) = \frac{\text{TP}(t)}{\text{TP}(t) + \text{FN}(t)} = \frac{\text{TP}(t)}{P}$$

The denominator does not depend on t at all: $\text{TP}(t) + \text{FN}(t)$ counts every positive in the data, whatever we predict about it.

So recall is $\text{TP}(t)$ divided by a constant, and TP is non-increasing. **Recall is non-increasing in t.**

Raise the threshold, recall can only fall or stay put. There are no exceptions and no conditions.

Precision has no such guarantee

$$\text{precision}(t) = \frac{\text{TP}(t)}{\text{TP}(t) + \text{FP}(t)}$$

Now **both** the numerator and the denominator move, and they move in the same direction. A ratio of two decreasing quantities can do anything at all.

> The denominator is the size of the set *we chose to flag*. It is not a property of the data; it is a property of our decision.

The one-step lemma

Raise t just enough to drop the lowest-scoring flagged instance. Write $T = \mathrm{TP}$, $F = \mathrm{FP}$, $n = T + F$ before the step, and let $y \in \{0, 1\}$ be the true label of the instance dropped.

$$\text{precision before} = \frac{T}{n} \qquad \text{precision after} = \frac{T - y}{n - 1}$$

Cross-multiplying, the step increases precision exactly when $n(T - y) > T(n - 1)$, that is when

$$y \;<\; \frac{T}{n} \;=\; \text{precision before}$$

And since y is 0 or 1

The instance dropped	y	Effect on precision
a false positive	0	**rises — always, since precision > 0 ✓**
a true positive	1	**falls — unless precision was already 1 ✗**

THE WHOLE RESULT, IN ONE SENTENCE

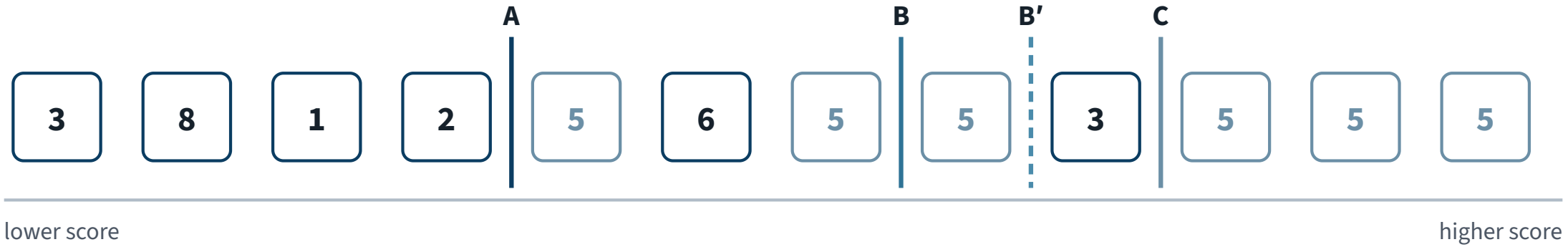
Raising the threshold past a **positive** lowers precision; raising it past a **negative** raises precision. The non-monotonicity is not an anomaly, it is the generic case.

Recall meanwhile falls by $1/P$ in the first case and by nothing in the second. Both metrics are decided by the same single fact: what the dropped instance was.

The textbook's counterexample, drawn

Twelve instances, ordered by score, and three thresholds

Everything to the right of a threshold is predicted "a 5". Green squares really are 5s.



threshold	flagged	precision	recall
A · low	8	6/8 = 75%	6/6 = 100%
B · central	5	4/5 = 80%	4/6 = 67%
B′ · one step right of B	**4**	**3/4 = 75%**	**3/6 = 50%**
C · high	3	3/3 = 100%	3/6 = 50%

✗ B to B′ is one step to the right, and precision falls from 4/5 to 3/4 while recall falls from 4/6 to 3/6.

Reading that off the lemma

> *"Starting from the central threshold and moving it just one digit to the right, precision goes from 4/5 (80%) down to 3/4 (75%)."*

- At B: $T = 4, n = 5$, precision $= 0.8$
- The instance dropped is a 5, so $y = 1$
- The lemma: $1 < 0.8$ is false, so precision does **not** increase — it falls to $3/4$

✓ And from B to C, two more steps, precision rises to 100% because the next instance dropped is a negative. Up, down, up: that is what "not monotone" means.

Now count it on sixty thousand instances

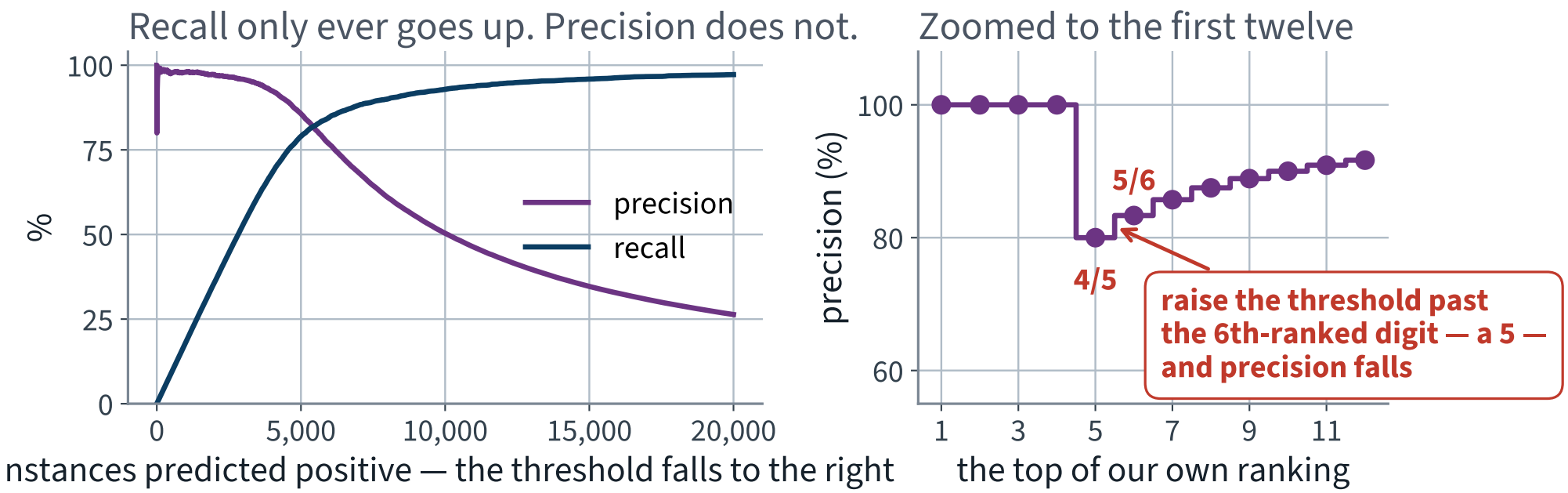
Sort every training instance by its out-of-fold score and walk the threshold down the list one instance at a time. That is 59,999 steps.

As the threshold rises, one step at a time	Steps
the instance dropped is **not** a 5 — precision rises	**54,579 ✓**
the instance dropped **is** a 5 — precision falls	**✗ 5,417**
… or stays at 1, because it was already 1	3

54,579 is the number of non-5s in the training set. And 5,417 + 3 = 5,420 is the number of 5s, less the one at the very top of the ranking, which has no step above it.

✓ Every single step is accounted for by the lemma. This is not a tendency; it is an exact classification of all 59,999 steps.

Both metrics, along our own ranking



The right-hand panel is the book's picture, on our data: the sixth-ranked digit is a 5, and dropping it takes precision from **5/6** to **4/5**.

What *is* guaranteed

- **Recall** is non-increasing in t. Always
- **The flagged count** $n(t)$ is non-increasing. Always
- **Precision**: nothing. It has no monotone envelope you can rely on, only a local rule

> **A REPAIR YOU WILL MEET AGAIN**
>
> Average precision is defined using the *maximum* precision at or above each recall level. That maximum is there for exactly one reason: to replace a non-monotone quantity by a monotone one. Lecture 14 builds mAP on top of it.

What the derivation buys you

- $\text{accuracy} = p\,\text{recall} + (1-p)\,\text{specificity}$
- Hence a classifier that never fires scores $1-p$, exactly, and $\partial\,\text{accuracy}/\partial\,\text{recall} = p$
- Recall is non-increasing in the threshold because its denominator is P, which the threshold cannot touch
- Precision's denominator is the flagged count, which the threshold chooses; one step raises precision iff the instance dropped is a negative
- Counted on our own data: 54,579 rises, 5,417 falls, 3 flat

Examinable: state and prove both claims; compute the never-fires accuracy from a base rate; apply the one-step lemma to a stated ranking.

APPLYING IT

What 96.90% was hiding

15 minutes

Apply that to our own number

Detector	Accuracy	Recall
Never fires	90.96%	✗ 0%
Ours	**96.90% ✓**	77.18%
Difference	5.94 points	77.18 points

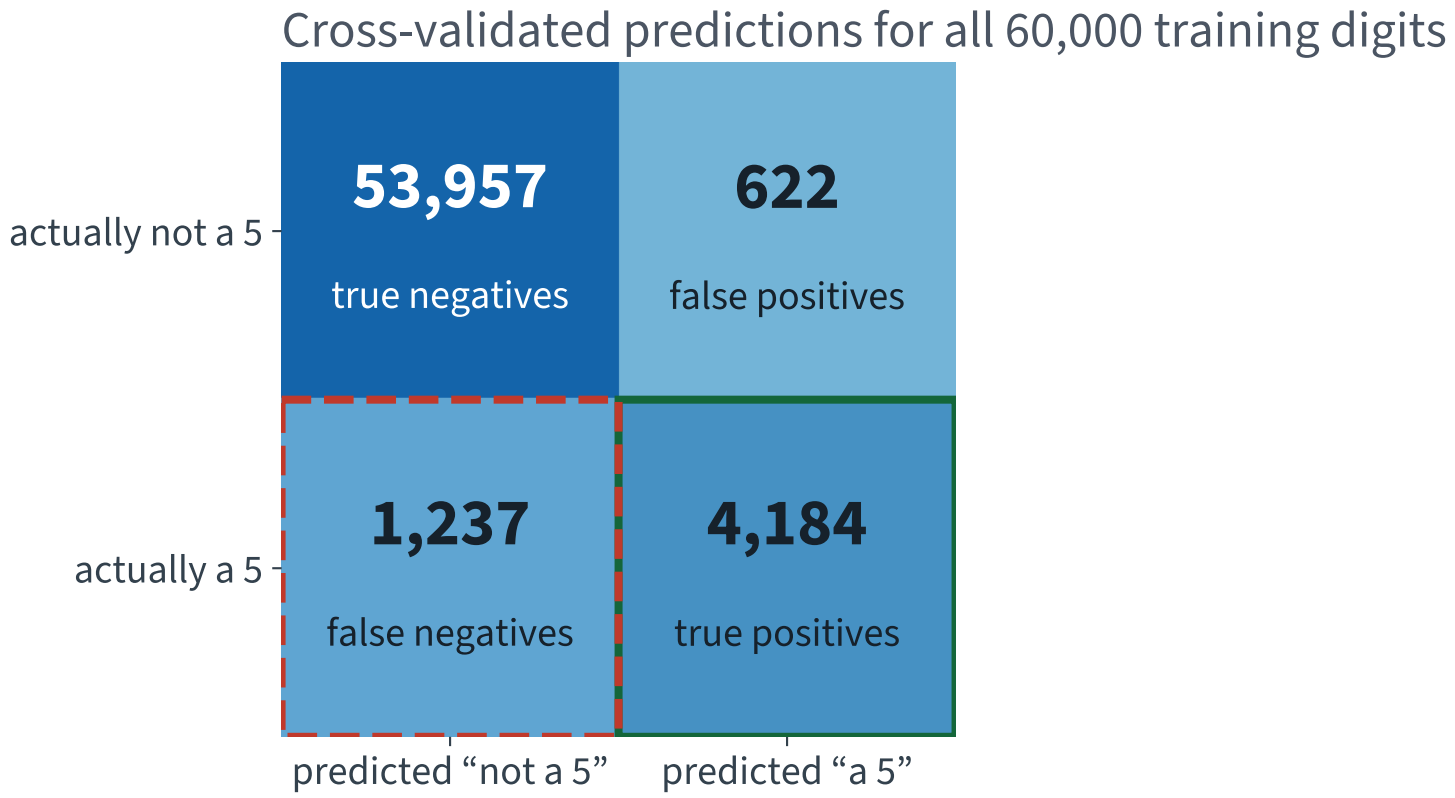
The identity says where those 5.94 points came from. Recall earns $p \times 77.18\%$, which is **6.97** points. Specificity fell from a perfect 100% to 98.86%, and that costs $(1 - p) \times 1.14\%$, which is **1.04** points.

$$5.94 \approx 6.97 - 1.04$$

The identity is exact; the three numbers on it are each rounded to two decimals, which is why they do not quite close on the page. 6.9733 − 1.0367 = 5.9366, and the measured difference is 5.9367.

✓ The arithmetic closes exactly. Now the question is whether 77.18% recall is good — and accuracy cannot answer it.

The same four numbers, laid out



✗ 1,237 fives went past unflagged — 22.8% of every 5 in the training set.

Reading a confusion matrix

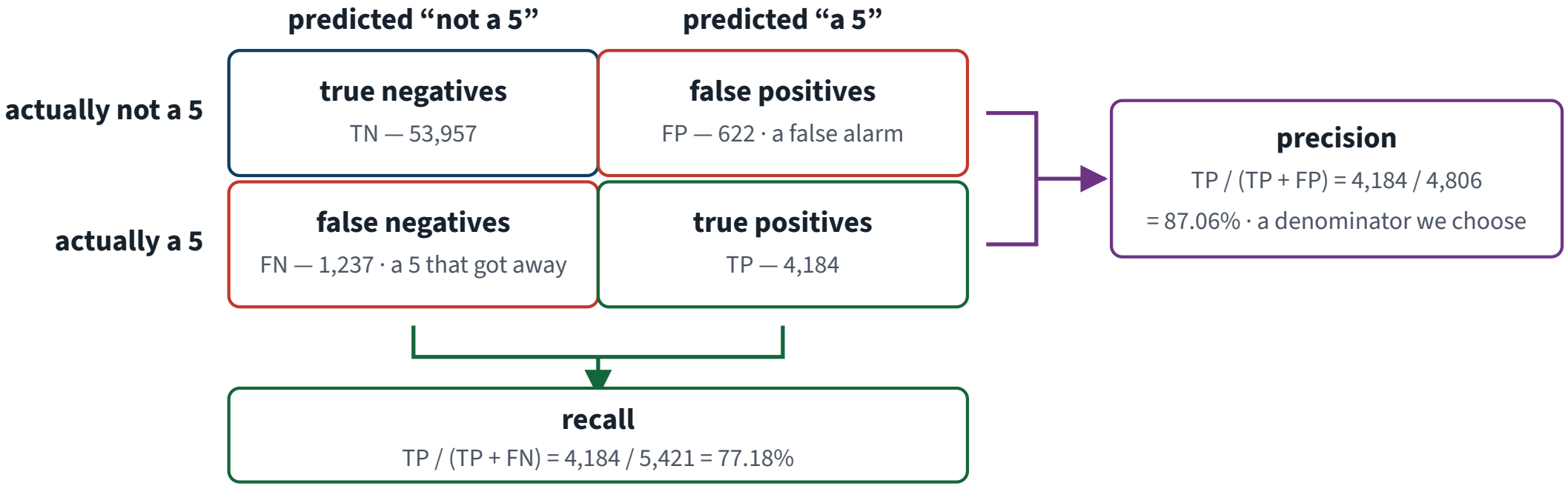
- Rows are the **actual** class; columns are the **predicted** class
- The diagonal is what went right; everything off it went wrong
- A perfect classifier has zeros off the diagonal, and nothing else
- Row 1 is the negatives: 53,957 correct, 622 false alarms
- Row 2 is the positives: 1,237 missed, 4,184 caught

Both mistakes are visible, separately, and they are not the same size or the same kind. Accuracy added them together and divided by 60,000.

Two ratios live in this table

One table, four counts, two different ratios

Precision divides down a column. Recall divides across a row. That is the whole difference.



Same numerator, different denominators. Precision divides by what we chose to flag; recall divides by what was there.

The two numbers accuracy was hiding

Metric	Value	Reads as
Precision	87.06%	of the digits we flag, 87% really are 5s
Recall	✗ 77.18%	of the 5s that exist, we find 77%
Accuracy	96.90%	—

✗ Nearly a quarter of the 5s go past the desk.

Both of those were computable an hour ago. Nothing new was fitted; we asked the same predictions a different question.

The two sentences, side by side

> “The detector is **96.9% accurate**.”
>
> True. Signed off. Deployed.

> “The detector **misses 22.8% of the 5s** — on the test shift, about 204 envelopes.”
>
> Also true. Same model, same data, same predictions.

Both are honest. Only one of them is a report the audit team can act on, and it is not the one the default metric produces.

The diagnosis

STATED ONCE, PRECISELY

Accuracy was not *wrong*. It correctly answered a question — "what fraction of all decisions were right?" — which nobody in this brief had asked.

- It weights the class we care about by $p = 0.09$
- It adds two errors that the client prices completely differently
- It has a floor of 90.96% that no model can go below by being cautious

✗ And we chose it deliberately, on the "metric before model" slide, before we had anything to optimise. Choosing early is necessary and it is not sufficient: this was still the wrong metric, and only the class counts would have said so.

THE METHOD

Precision, recall, and the curves

40 minutes · examinable

Fix 1 · ask for both numbers

```python
from sklearn.metrics import precision_score, recall_score, f1_score

precision_score(y_train_5, y_pred)   # 0.8706  — 4184 / (4184 + 622)
recall_score(y_train_5, y_pred)      # 0.7718  — 4184 / (4184 + 1237)
f1_score(y_train_5, y_pred)          # 0.8182
```

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Recall is also called *sensitivity* or the *true positive rate*. Three names, one quantity; the third one comes back on the ROC curve in twenty minutes.

Fix 2 · one number, when you must have one

F_1 is the **harmonic** mean of precision and recall:

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Ours is **81.82%**, which sits below both of the numbers it combines. That is not an accident.

Why harmonic and not ordinary

Precision	Recall	Arithmetic mean	F_1
0.90	0.90	0.900	0.900
0.99	0.81	0.900	0.891
1.00	0.02	✗ 0.510	0.039 ✓

The harmonic mean is dominated by the *smaller* of the two. A classifier that flags three digits a year and is right every time has an arithmetic mean above one half and an F_1 near zero.

AND THAT IS A CHOICE, NOT A VIRTUE

F_1 rewards classifiers with **similar** precision and recall. Our brief does not ask for similar precision and recall. It asks for the 5s to be found, within a fixed desk capacity.

Which one you want depends on the brief

A filter for children’s videos

Rejecting good videos is annoying. Letting a bad one through is not.
✓ **High precision**, low recall — and a human reviews the rest.

A shoplifter detector for a surveillance system

A guard checking a false alarm costs a minute. A missed thief costs the stock. ✓ **High recall**, and accept the false alarms.

✗ Ours is the second shape, with a hard cap: find the 5s, and do not send the desk more than it can process.

Both examples are the textbook’s. They are worth keeping because they are memorable and because they point in opposite directions.

Fix 3 · you cannot have both

Raising recall lowers precision, and lowering recall raises it — usually.

“Usually” is doing real work in that sentence, and the one-step lemma says exactly how much: precision rises at 54,579 of the steps and falls at 5,417 of them. The *trend* is reliable; the individual step is not.

> **THE PRECISION/RECALL TRADE-OFF**
>
> It is not a property of our model, or of SGD, or of MNIST. It is a property of ranking instances and cutting the ranking somewhere.

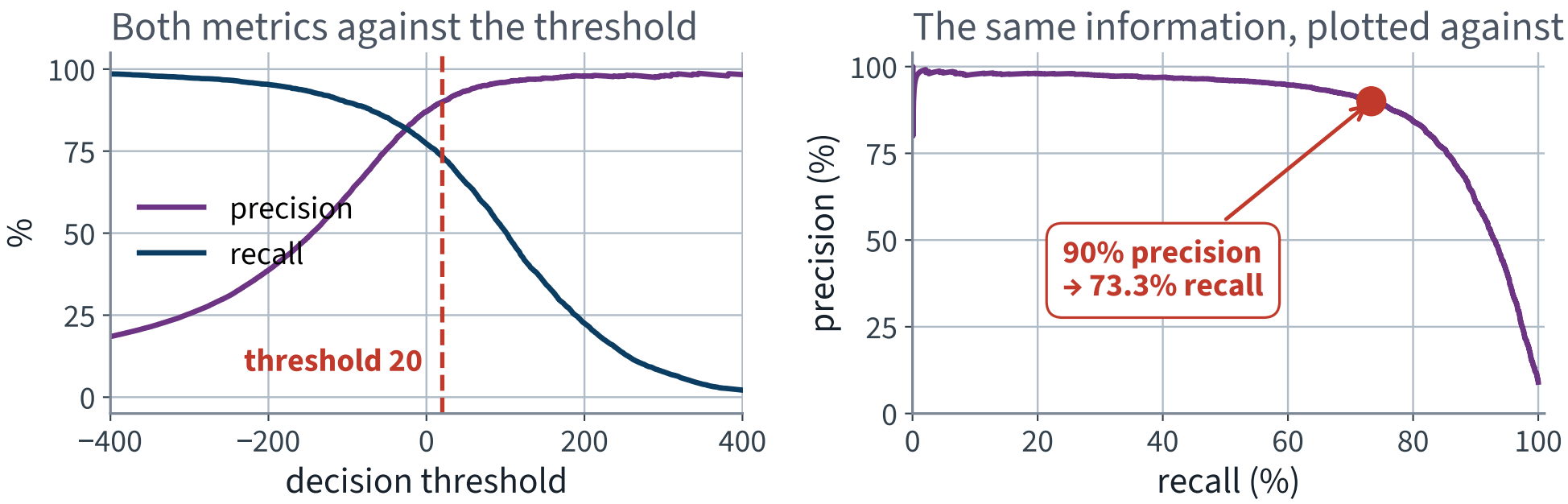
Where the dial is

```
y_scores = clf.decision_function([some_digit])
print(y_scores)                          # e.g. array([1231.4])

threshold = 0
y_scores > threshold                     # array([ True])

threshold = 3000
y_scores > threshold                     # array([False]) — the same 5, missed
```

Every scoring classifier computes a number and compares it to a threshold. `predict()` hides the number and hard-codes the threshold at zero.

✓ There is no `set_threshold()` method, and there should not be. You compute the scores and you compare them yourself.

The dial, and the curve it traces



Precision starts to fall off a cliff at about 80% recall. Choose the operating point just before the cliff, not on it.

Fix 4 · choose the threshold deliberately

```
import numpy as np

flagged = recalls[:-1] * n_pos / precisions[:-1]   # tp / precision = tp + fp
ok  = (precisions[:-1] >= 0.90) & (flagged >= 500)
idx = int(np.argmax(ok))                           # first crossing WITH support
threshold_90 = thresholds[idx]                     # 20.09

y_pred_90 = (y_scores >= threshold_90)
print(precision_score(y_train_5, y_pred_90))   # 0.9001
print(recall_score(y_train_5, y_pred_90))      # 0.7333
```

`argmax` on a boolean array returns the first `True`. The idiom is compact and it is not obvious; read it once and remember it.

✗ Ninety per cent precision costs us 3.86 points of recall, from 77.18% down to 73.33%.

Why `& (flagged >= 500)` is in that line

We have just spent twenty minutes proving this curve is **not monotone**. So “the first index reaching 90%” could be a single lucky step, held up by a handful of flagged digits, that the very next threshold falls back below.

Requiring support turns the crossing into a claim you can defend. And on this data it **passes**:

	threshold	digits flagged
first crossing, unguarded	20.09	4,416
first crossing with support ≥ 500	20.09	4,416

✓ Same point — the result, not a reason to delete the guard. You now *know* it rests on 4,416 digits.

> Note the asymmetry with the recall constraint two slides on, which correctly uses `np.where(...)[0][-1]` — the *last* index. For a floor on recall you want the highest threshold that still clears it; for a floor on precision, the lowest. Getting these the same way round is a common and silent error.

Two traps in that one line

DO NOT CHOOSE THE THRESHOLD ON THE TEST SET

A threshold is a hyperparameter. Choosing it by looking at test performance is the same error as choosing α that way, and it produces the same optimistic bias. Choose it on cross-validated training scores, then measure once.

And the second: aiming for very high precision is nearly free-looking and very expensive. At **99%** precision our recall falls to **2.12%** — a detector that finds one 5 in fifty and is almost never wrong when it does.

"99% precision" is a sentence that should always be followed by "at what recall?"

✗ — and then by "on how many instances?" Apply the support test from the previous slide and this point fails it: 99% precision is reached on only 116 flagged digits, across a plateau 21 thresholds wide. Compare the 90% point: 4,416 digits and 4,411 thresholds.

> Which is why 2.12% is quoted here as an *illustration* of what high precision costs, and never proposed as an operating point. The right response to a target you cannot meet with support is to say so — not to lower the support test until it passes.

Fix 5 · the ROC curve

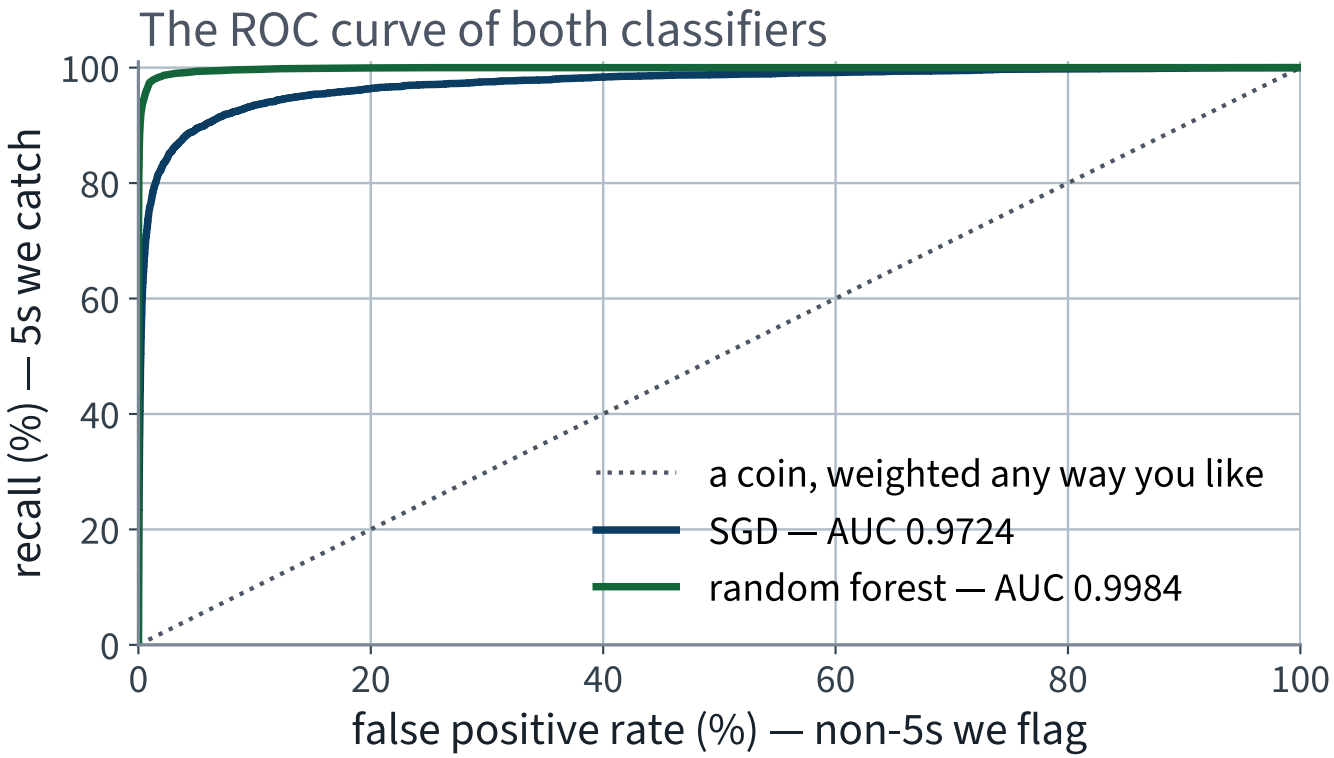
Same sweep, different pair of axes: the true positive rate against the false positive rate.

$$\mathrm{TPR} = \frac{\mathrm{TP}}{P} = \text{recall}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{N} = 1 - \text{specificity}$$

```
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
roc_auc_score(y_train_5, y_scores)          # 0.9724
```

Note that **both** denominators are fixed by the data. Neither axis moves when we change the threshold's effect on the flagged count — which is precisely why the ROC curve behaves so much better than the PR curve, and why that is a warning rather than a recommendation.

Two classifiers on the same axes



The diagonal is a coin. The area under the curve is 0.5 for the coin, 1.0 for a perfect classifier, and 0.9724 for ours.

What the area actually measures

> ROC AUC is the probability that a randomly chosen positive is scored above a randomly chosen negative. It is a statement about the **ranking** and about nothing else.

- It does not depend on the threshold — there is no threshold in it
- It does not depend on the base rate, because both its axes are normalised within a class
- So it cannot tell you whether the classifier is usable. Only how well it ranks

✗ That base-rate independence is usually sold as a virtue. On a rare-event problem it is the whole difficulty.

Fix 6 · which curve to prefer

> THE RULE
>
> *Prefer the precision/recall curve whenever the **positive class is rare**, or whenever you care more about false positives than about false negatives. Otherwise use the ROC curve.*

The reason is arithmetic. FPR divides false positives by N, which is huge when positives are rare, so a large number of false alarms produces a small, comfortable-looking FPR. Precision divides the same false positives by the flagged count, which is small.

✓ Do not take that on trust. Measure it.

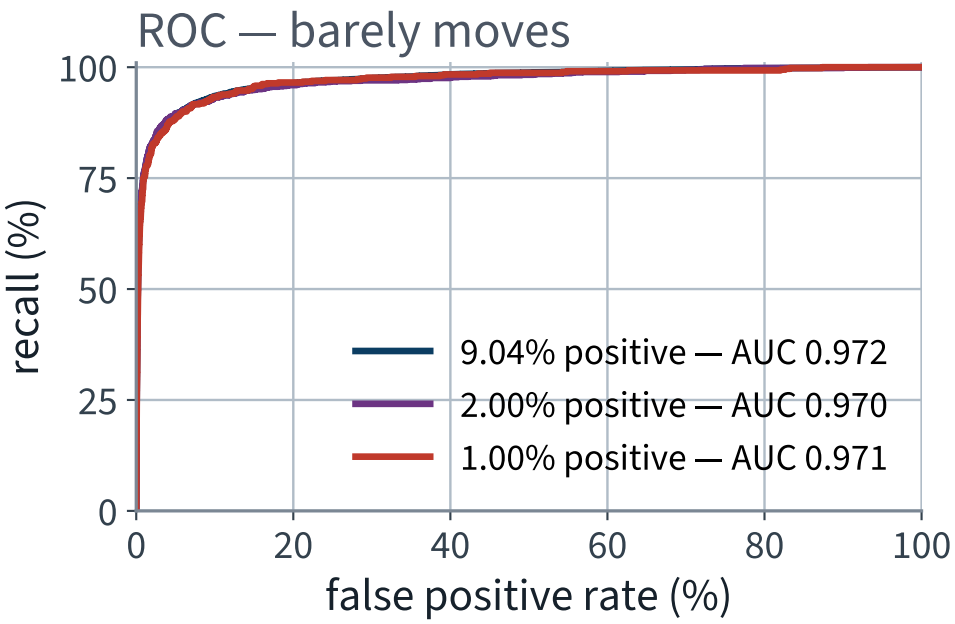
The same classifier, three base rates

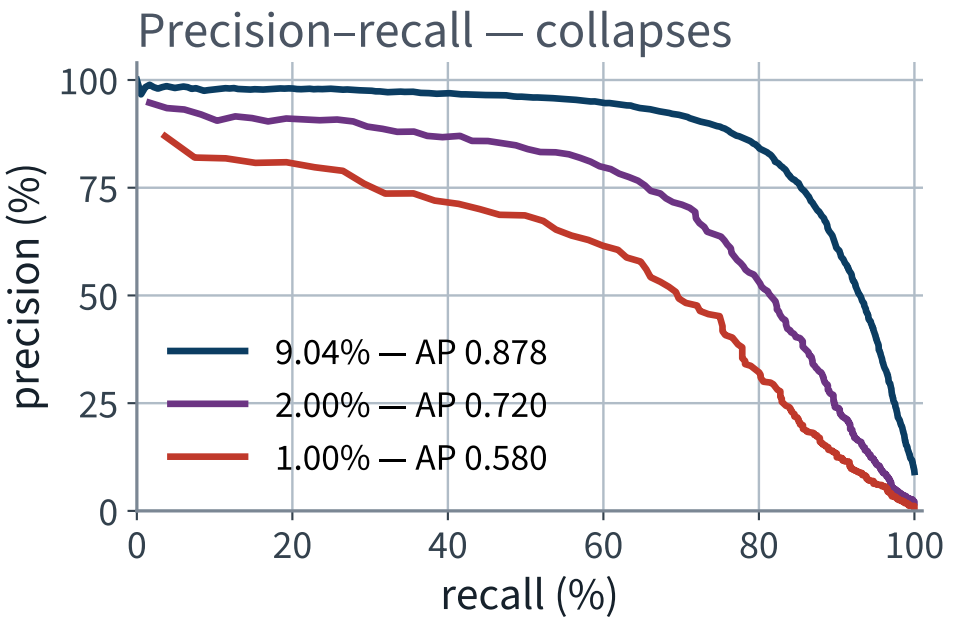
Take our scores unchanged, and thin the positive class to make the event rarer. The model, the ranking and the scores are *identical*; only the class balance moves.

Base rate	Positives	ROC AUC	Average precision
9.04% — as measured	5,421	0.9724	0.8784
2.00%	1,114	0.9699	0.7203
1.00%	551	**0.9709 ✓**	**✗ 0.5800**

✗ ROC AUC moves by 0.0015. Average precision falls by 0.298.

The same three experiments, drawn





✗ Three indistinguishable ROC curves and three obviously different PR curves, from one set of scores. The ROC curve cannot see the problem you have.

ROC on a rare positive class

FAILURE CONDITION

Three indistinguishable ROC curves and three obviously different precision/recall curves, from one set of scores. The false-positive rate divides by the abundant class, so the ROC curve cannot see what a rare positive costs — and it looks reassuring while it fails to.

Fix 7 · and now a better classifier

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=100, random_state=42)

y_proba = cross_val_predict(forest, X_train, y_train_5, cv=cv,
                            method="predict_proba")
y_scores_forest = y_proba[:, 1]        # the positive class column
```

A DIFFERENT DIAL

A forest has no `decision_function`. It has `predict_proba`, which returns one column per class. The second column — the estimated probability of the positive class — plays exactly the role of the score, and every curve above works unchanged.

The same four metrics, on the forest

Metric	SGD	Random forest
Accuracy	96.90%	98.77%
Precision	87.06%	98.93%
Recall	77.18%	**87.31% ✓**
F_1	81.82%	92.76%
ROC AUC	0.9724	0.9984
Average precision	0.8784	0.9883
Recall available at 90% precision	73.33%	**97.51% ✓**

Accuracy improves by 1.87 points, which reads as polish. The last row improves by **24.18 points**, and it is the row the brief is about.

Which metric noticed?

- **Accuracy**: 96.90% → 98.77%. Both look excellent; the difference looks like polish
- **Recall**: 77.18% → 87.31%. 10.13 points of the thing we care about
- **Recall at 90% precision**: 73.33% → 97.51%. Two completely different systems

✗ Three descriptions of one comparison. Which one you report decides whether anyone approves the change.

Report the one that matches the brief, and say why you chose it. That sentence is the deliverable.

THE LAST FIFTEEN MINUTES

An operating point, and beyond binary

15 minutes

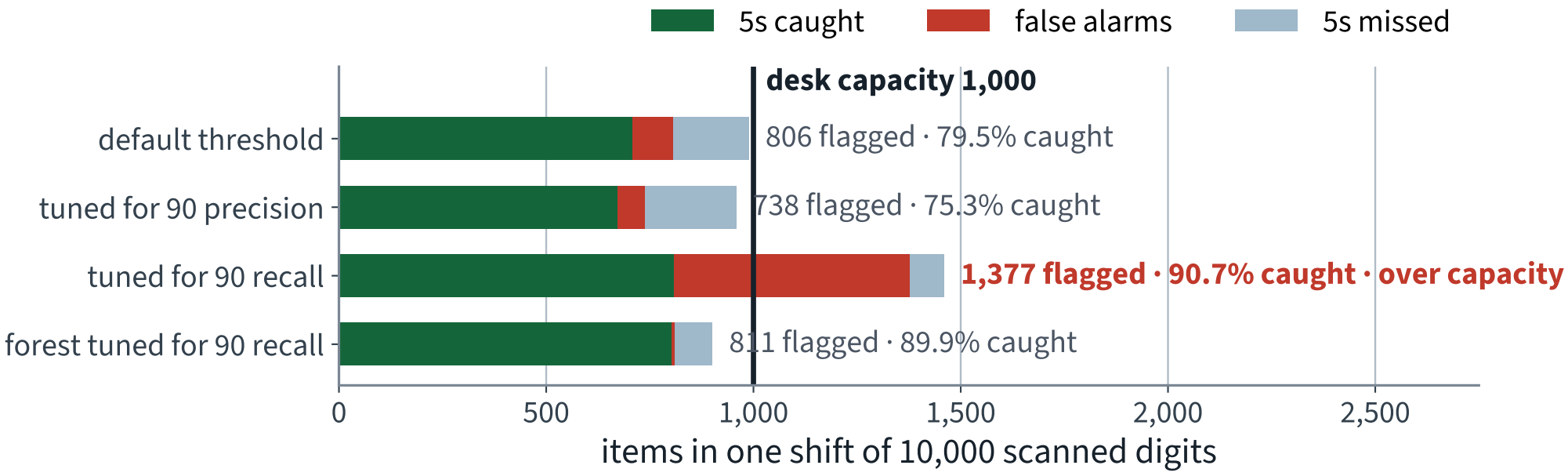
The operating point, stated

> **WHAT THE CLIENT ASKED FOR, IN OUR VOCABULARY**
>
> **Catch at least 90% of the 5s** on a shift, and **flag no more than 1,000 items**, out of 10,000 scanned.

Two constraints, one dial. A threshold either satisfies both or it does not, and the honest answer may be that none does.

✓ Choose the threshold on the cross-validated training scores. Then touch the test shift once.

Four operating points, counted



✓ The forest, tuned for the same 90% recall, flags 811 — inside capacity, because almost none of them are false alarms.

Now defend the choice

Whichever threshold you propose, you must be able to answer all four:

1. Which constraint did you satisfy, and which did you sacrifice?
2. What does one extra point of recall cost, in items sent to the desk?
3. On what data was the threshold chosen — and had you seen the test shift when you chose it?
4. What happens to your answer if the base rate changes next quarter?

> **THE DEFENSIBLE REPORT**
>
> “The forest at threshold 0.44 catches 89.9% of the 5s and flags 811 items per shift. It misses the 90% target by 0.1 points, which is inside the variation between the folds it was chosen on.”

The wrinkle in that sentence

We chose 0.44 because it gave **90.39%** recall on the cross-validated training scores. On the test shift it gives **89.91%**.

Random forest at threshold 0.44	Recall	Precision	Flagged
Cross-validated, on the training set	90.39%	98.33%	—
Test shift, measured once	89.91%	98.89%	811

Half a point below target. Is that a failure, a bad fold, or noise? It is noise: the target was hit on a different sample, and a threshold chosen at exactly 90% will land either side of it about half the time.

✗ Report it, do not tune it away. Re-tuning to make the test number reach 90% is fitting the test set, and it is the temptation Lecture 2 told you to refuse.

The general rule

> **NEVER OPTIMISE ONE METRIC OF A PAIR**
>
> Precision and recall, bias and variance, latency and accuracy, memory and throughput. Ask for one and you will get it, at the price of the other, and the price will not be in the reply.

The counter-measure is not cleverness. It is stating the second quantity as a **constraint** in the same sentence as the first.

The rest of Chapter 3, in four minutes

Everything above was one class against the rest. The chapter covers three more shapes, and all of them reuse the same confusion matrix.

Shape	Output per instance	Example
Binary	one label from two	is it a 5?
Multiclass	one label from many	which digit is it?
Multilabel	several labels at once	is it large? is it odd?
Multioutput	several labels, each multiclass	the denoised image, pixel by pixel

Multiclass, from binary parts

- **One-versus-rest**: ten detectors like ours, and take the highest score. Ten fits
- **One-versus-one**: one detector per pair, and take the most-won duel. $10 \times 9/2 = 45$ fits, each on a much smaller training set
- Scikit-learn picks one for you automatically when you hand a multiclass target to a binary classifier, and does not mention which

✗ Another default nobody asked for. It matters: for algorithms that scale badly with training-set size, forty-five small fits beat ten large ones.

Multilabel and multioutput

- **Multilabel**: the target is a vector of booleans. Metrics are computed per label and then averaged — `average="macro"` weights every label equally, `average="weighted"` weights by support
- **Multioutput**: each output is itself multiclass. Removing noise from an image is 784 separate 256-class problems, one per pixel

✗ The averaging argument is a third default that changes the number without changing the model. On imbalanced labels, macro and weighted averages can disagree substantially.

All of this is Chapter 3 and all of it is examinable. None of it changes the lesson: name the metric, and name the average.

Back to the specimen question — 1

A colleague reports 99.2% accuracy on a rare defect, correctly cross-validated. *State one further quantity you would need, and why.*

> **WHAT EARNS THE MARKS**
>
> The **base rate**. Without it, 99.2% cannot be located relative to the classifier that never fires, which scores $1 - p$ and requires no model. At $p = 0.009$ that baseline is 99.1%, so the reported result is one tenth of a point above doing nothing.

Also good answers: the confusion matrix, precision and recall together, or the recall alone — each of them locates the number against something. Weaker: “the test set size” — relevant, but it bounds the precision of the estimate rather than its meaning.

Specimen exam question — Part B

A ranking classifier is evaluated on 10 instances, of which 4 are positive. Sorted by score, highest first, the true labels are

1, 1, 0, 1, 0, 0, 1, 0, 0, 0

1. Compute precision and recall when the top 4 are flagged, and when the top 5 are flagged.
2. The threshold is raised from “top 5” to “top 4”. State, with the one-step lemma, whether precision rises or falls, and why.

Three marks. Worked answer on the next slide.

Specimen answer — Part B

Flagged	TP	Precision	Recall
top 4	3	3/4 = 75%	3/4 = 75%
top 5	3	3/5 = 60%	3/4 = 75%

Going from top 5 to top 4 drops the fifth-ranked instance, whose label is **0**. The lemma says precision increases exactly when $y < \text{precision before}$, and $0 < 0.6$, so precision ✓ **rises**, from 60% to 75%. Recall is unchanged, because no positive was dropped.

✗ The common wrong answer is "precision falls, because raising the threshold always loses information". Both halves of that sentence are wrong, and the lemma is three lines.

What is examinable from today

- Framing a detection task as binary classification, and what that does to the class balance
- Accuracy: the definition, and computing it for a stated confusion of counts
- Why a trivial baseline must be measured before a result is interpreted, and computing it from a base rate
- Cross-validation for classifiers, and why the folds are stratified
- Why a score computed on the training set is not a generalisation estimate

Not examinable: Colab mechanics, wall-clock timings, the internals of `fetch_openml`.

Making today reproducible

- One seed, written down: `random_state=42` on the estimator and on the splitter
- `shuffle=True` stated explicitly, because the default is `False` and two students with differently ordered frames would otherwise get different folds
- The fold scores saved, not just the mean — you cannot recover a spread from an average
- Library versions printed in the first cell, so a number that moves next year has somewhere to start

✗ This is graded. A result nobody else can reproduce is not a result.

Version pinning and Colab mechanics are engineering practice, outside the book and **not examinable**.

The notebook for this lecture

Open Lecture 3 in Colab

- **Runs on free CPU.** The slowest cell is the cross-validated fit, about 30 seconds
- Everything on today's slides: the split, the never-fires baseline, the detector, the confusion matrix, the curves, and the threshold choice
- It verifies the two results of the mathematics numerically — the accuracy identity closes to the last decimal, and the one-step lemma is checked at every one of the 59,999 thresholds

> **WHAT TO CHANGE BEFORE THE NEXT LECTURE**
>
> Detect a different digit. Change one character — the `5` in `y_train == 5` — and re-run. The baseline moves, the accuracy moves with it, and the pair tells you something neither does alone.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

What we did

1. Framed a detection task as binary classification, which makes the positive class rare by construction — one in ten here, one in ten thousand for fraud, and the arithmetic is the same
2. Derived $\mathrm{accuracy} = p\,\mathrm{recall} + (1-p)\,\mathrm{specificity}$: a classifier that never fires scores exactly $1-p$, and accuracy weights the class you care about by how rare it is
3. Proved that recall is monotone in the threshold and precision is not, and that the one-step lemma says exactly when each moves
4. Read the confusion matrix, and separated the two errors that accuracy adds together and the client prices differently
5. Used precision, recall and F1 — and saw what each is blind to
6. Chose an operating point against a stated constraint, and defended it with the PR curve rather than the ROC curve, because the negatives outnumber the positives ten to one
7. Extended to multiclass, multilabel and multioutput, where the same matrix and the same ratios still apply

In the book

Topic	Chapter
Confusion matrix, precision, recall, F_1	3
The precision/recall trade-off and `decision_function`	3
The ROC curve and ROC AUC; when to prefer PR	3
Multiclass, error analysis, multilabel, multioutput	3
Random forests and `predict_proba`	3, 6
Average precision, and the maximum that repairs it	12 (Lecture 14)

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 3

five, in the style of the written paper

Exercises — Lecture 3

1 On the MNIST 5-detector, a classifier that never fires scores 90.96% accuracy on the training set. State what that number is a measurement of. [3]

2 Prove or disprove: as the decision threshold is lowered, precision increases monotonically. [5]

3 A client asks for “90% precision”. Explain why that is not yet a specification, and give the one thing you would add. [4]

Solutions on Lecture 4’s deck.

Exercises — Lecture 3 (continued)

4 Two models have the same ROC AUC. One is far better on a precision–recall curve. What does that tell you about the problem? [4]

5 The confusion matrix of the never-fires classifier has two zero cells. Name them, and say which single metric would have exposed the model immediately. [4]

Solutions on Lecture 4's deck.

THE FIVE SET AT THE END OF LECTURE 2

Solutions Lecture 2

not part of this lecture

Solutions — Lecture 2

1. The normal equation is $\hat{\theta} = (X^\top X)^{-1} X^\top y$. Your design matrix has a column…

✓ $X^\top X$ is singular; the pseudoinverse returns the minimum-norm least-squares solution.

- A linear dependence among columns makes X rank-deficient, so $X^\top X$ has a zero eigenvalue and no inverse
- The fitted *values* are still unique — the projection onto the column space is — only the coefficients are not

2. A pipeline scales the features and then fits a ridge model, and is passed whole to cross_val_score. Explain…

✓ The scaler would be fitted on every fold's validation rows, so each fold's score would be optimistic.

- A scaler is a fitted object: its mean and variance are parameters learned from data
- Fitting it before the split lets validation rows influence the transformation applied to themselves

Solutions — Lecture 2 (2)

3. You report a 95% confidence interval for the test RMSE. A colleague says the interval proves the model is…

✓ Was the baseline scored on the same test rows?

- An interval describes the uncertainty of *one* estimate, not the difference between two
- Comparing two systems needs their difference's spread, which needs them measured on the same rows

4. k-fold cross-validation reports a mean of 0.71 with folds $[0.52, 0.79, 0.75, 0.74, 0.75]$. What do you…

✓ One fold is an outlier; investigate it rather than reporting the mean.

- Four folds agree closely and one does not, which is a property of the data in that fold, not noise around a common value
- The mean of a bimodal set of scores describes none of them

Solutions — Lecture 2 (3)

5. Explain, in two sentences, why the test set may be looked at exactly once, and what you may legitimately do…

✓ Looking at it more than once makes it a validation set. If it disappoints, you may report it — not tune against it.

- Every decision taken after seeing the test score is a decision the score no longer measures honestly
- The permitted response is a new experiment on new held-out data, or an honest report of what was found

LECTURE 4 · GÉRON CH. 4

Training models

Gradient descent, derived — batch, mini-batch and stochastic, and what the learning rate does
Linear regression by descent rather than by the normal equation
Logistic and softmax regression, and reading a coefficient as log-odds

Run the Lecture 3 notebook first